

Canny Edge Detection

GNR602: Advanced Methods in Satellite Image Processing

Team Members:

Niranjana Veram (210020085)
Aditya Kumar (210020005)
Bhavani Shankar (210100043)

1 Introduction

Edge detection is a critical step in many image processing tasks, particularly in computer vision, feature extraction, and image analysis. The goal of edge detection is to identify the boundaries within an image, which represent significant changes in intensity or color. The Canny edge detection algorithm, introduced by John F. Canny in 1986, is widely considered one of the most popular and effective edge detection methods due to its ability to detect a wide range of edges with low error rates.

The Canny edge detector involves several steps:

- Smoothing the image using a Gaussian filter.
- Calculating the gradient magnitude and direction.
- Applying non-maximum suppression to thin out edges.
- Performing double thresholding to classify edge pixels into strong, weak, and non-edges.
- Performing edge tracking by hysteresis to remove spurious edges.

In this report, we will delve into the significance of the parameters in the Canny edge detector, namely the *sigma* (used for Gaussian blurring) and the *low* and *high* thresholds that control the edge detection process.

2 Importance of Parameters in Canny Edge Detection

2.1 Sigma

The *sigma* parameter controls the amount of blurring applied to the image before edge detection. It is used in the Gaussian filter, which smoothens the image to remove noise. The equation for the Gaussian kernel used for blurring is given by:

$$G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} \quad (1)$$

Where: - x and y are the coordinates of the pixel relative to the kernel center. - σ is the standard deviation, which controls the spread of the Gaussian function. A higher value of σ results in a larger kernel, causing more blurring.

A high *sigma* value results in more blurring, making it more difficult to detect sharp edges, while a low *sigma* value retains more of the finer details but may also preserve more noise.

2.2 Lower and Upper Thresholds

The Canny edge detection algorithm uses two thresholds to classify pixels as strong edges, weak edges, or non-edges. The *lower threshold* determines the minimum edge strength that will be considered a weak edge, while the *upper threshold* sets the minimum strength for strong edges. Pixels that have a magnitude greater than the upper threshold are considered strong edges, while pixels with values between the two thresholds are considered weak edges. Pixels below the lower threshold are discarded.

The edge classification can be defined mathematically as:

$$\text{Edge}(x, y) = \begin{cases} \text{Strong Edge,} & \text{if } M(x, y) > T_{\text{high}} \\ \text{Weak Edge,} & \text{if } T_{\text{low}} \leq M(x, y) \leq T_{\text{high}} \\ \text{Non-edge,} & \text{if } M(x, y) < T_{\text{low}} \end{cases}$$

Where: - $M(x, y)$ is the gradient magnitude at pixel (x, y) . - T_{high} and T_{low} are the high and low threshold values, respectively.

3 Explanation of the Code

Below is a breakdown of the code used for the Canny edge detection:

3.1 Gaussian Kernel Function

The Gaussian filter is applied to the image for blurring. The kernel is generated based on the *sigma* value:

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{(x - \frac{s-1}{2})^2 + (y - \frac{s-1}{2})^2}{2\sigma^2}\right)$$

```
def gaussian_kernel(size, sigma):
    kernel = np.fromfunction(
        lambda x, y: (1 / (2 * np.pi * sigma ** 2)) * np.exp(-((x - (size - 1) / 2) ** 2 + (y - (size - 1) / 2) ** 2) / (2 * sigma ** 2)),
        (size, size)
    )
    return kernel / np.sum(kernel)
```

This is an implementation of Equation (1), where the Gaussian kernel is calculated for smoothing the image.

3.2 Gaussian Blurring

The Gaussian blurring is applied to the image by convolving the image with the Gaussian kernel. In the code, the Gaussian kernel is created with a size proportional to σ , and then the image is convolved with this kernel:

```
def apply_gaussian_blur(img, sigma):
    kernel_size = int(2 * np.ceil(3 * sigma) + 1)
    kernel = gaussian_kernel(kernel_size, sigma)
    return convolve(img, kernel)
```

This convolution operation helps in reducing the high-frequency noise and smoothens the image before detecting edges.

3.3 Sobel Filters

The Sobel operator calculates the gradient of the image at each pixel, which helps identify edges. The gradient is calculated in both the x and y directions. The gradient magnitudes are computed as:

$$G_x(x, y) = \sum_{i=-1}^1 \sum_{j=-1}^1 f(x+i, y+j) G_x(i, j) \quad (2)$$

$$G_y(x, y) = \sum_{i=-1}^1 \sum_{j=-1}^1 f(x+i, y+j) G_y(i, j) \quad (3)$$

Where: - $G_x(i, j)$ and $G_y(i, j)$ are the Sobel kernels for horizontal and vertical gradient calculation, respectively. - $f(x, y)$ is the image pixel value.

The gradient magnitude is then computed as:

$$M(x, y) = \sqrt{G_x(x, y)^2 + G_y(x, y)^2}$$

And the gradient direction is:

$$\theta(x, y) = \text{atan2}(G_y(x, y), G_x(x, y))$$

```

def sobel_filters(img):
    Gx = np.array([[[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]]])
    Gy = np.array([[[-1, -2, -1], [0, 0, 0], [1, 2, 1]]])

    grad_x = convolve(img, Gx)
    grad_y = convolve(img, Gy)

    grad_magnitude = np.hypot(grad_x, grad_y)
    grad_angle = np.arctan2(grad_y, grad_x) * (180.0 / np.pi) % 180 # Convert to degrees

    return grad_magnitude, grad_angle

```

3.4 Non-Maximum Suppression

Non-maximum suppression thins out the edges to make them more distinct. This process looks at the local maximum along the gradient direction and suppresses pixels that are not part of the edge. Given the gradient direction and magnitude $\theta(x, y)$, for each pixel, we compare it to its neighbors in the direction of the gradient. If the gradient magnitude is smaller than its neighbors in that direction, it is suppressed.

Mathematically, non-maximum suppression can be described as:

$$\text{Suppressed}(x, y) = \begin{cases} M(x, y), & \text{if } M(x, y) \text{ is local maximum along direction } \theta(x, y) \\ 0, & \text{otherwise} \end{cases}$$

Where: - $M(x, y)$ is the gradient magnitude at pixel (x, y) . - $\theta(x, y)$ is the gradient direction at pixel (x, y) .

```

def non_maximum_suppression(magnitude, angle):
    rows, cols = magnitude.shape
    output = np.zeros_like(magnitude)
    angle = angle / 180.0 * np.pi # Convert to radians

    for i in range(1, rows - 1):
        for j in range(1, cols - 1):
            # Get the direction of the gradient
            direction = angle[i, j]

            # Determine the neighbors
            if (0 <= direction < np.pi / 8) or (15 * np.pi / 8 <= direction <= 2 * np.pi):
                neighbor1, neighbor2 = magnitude[i, j + 1], magnitude[i, j - 1]
            elif (np.pi / 8 <= direction < 3 * np.pi / 8):
                neighbor1, neighbor2 = magnitude[i + 1, j - 1], magnitude[i - 1, j + 1]
            elif (3 * np.pi / 8 <= direction < 5 * np.pi / 8):
                neighbor1, neighbor2 = magnitude[i + 1, j], magnitude[i - 1, j]
            else:
                neighbor1, neighbor2 = magnitude[i - 1, j - 1], magnitude[i + 1, j + 1]

            # Suppress non-maximal edges
            if magnitude[i, j] >= neighbor1 and magnitude[i, j] >= neighbor2:
                output[i, j] = magnitude[i, j]

    return output

```

3.5 Double Thresholding and Edge Tracking

Double thresholding helps classify the edges into strong and weak categories, and edge tracking ensures that weak edges connected to strong edges are retained. The process of edge tracking is mathematically described by:

$$\text{Final Edge}(x, y) = \begin{cases} 1, & \text{if } M(x, y) > T_{\text{high}} \\ 1, & \text{if } M(x, y) \text{ is weak edge and connected to strong edge} \\ 0, & \text{if } M(x, y) < T_{\text{low}} \end{cases}$$

Where: - $M(x, y)$ is the gradient magnitude at pixel (x, y) . - T_{high} and T_{low} are the high and low thresholds.
 - Weak edges are retained if they are connected to strong edges.

```

def double_threshold(magnitude, low_thresh, high_thresh):
    strong_edges = (magnitude > high_thresh)
    weak_edges = (magnitude >= low_thresh) & (magnitude <= high_thresh)

    return strong_edges, weak_edges

def edge_tracking(strong_edges, weak_edges):
    rows, cols = strong_edges.shape
    output = np.copy(strong_edges)

    for i in range(1, rows - 1):
        for j in range(1, cols - 1):
            if weak_edges[i, j]:
                if ((strong_edges[i + 1, j] or strong_edges[i - 1, j]) or
                    (strong_edges[i, j + 1] or strong_edges[i, j - 1]) or
                    (strong_edges[i + 1, j + 1] or strong_edges[i - 1, j - 1])):
                    output[i, j] = 1

    return output

```

4 Results

In this section, we present the results of applying the Canny edge detection algorithm with different values of *sigma*, *low threshold*, and *high threshold*. The impact of these parameters on the edge detection process will be demonstrated.

4.1 Effect of Sigma

The *sigma* value determines how much blurring is applied to the image. A higher *sigma* value results in more blurring, which can smooth out finer details, while a lower *sigma* value preserves more fine details, but may also preserve more noise.

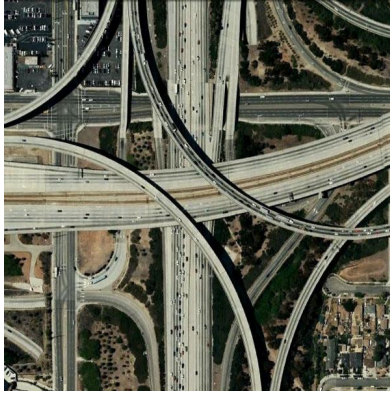


Figure 1: Original Image.

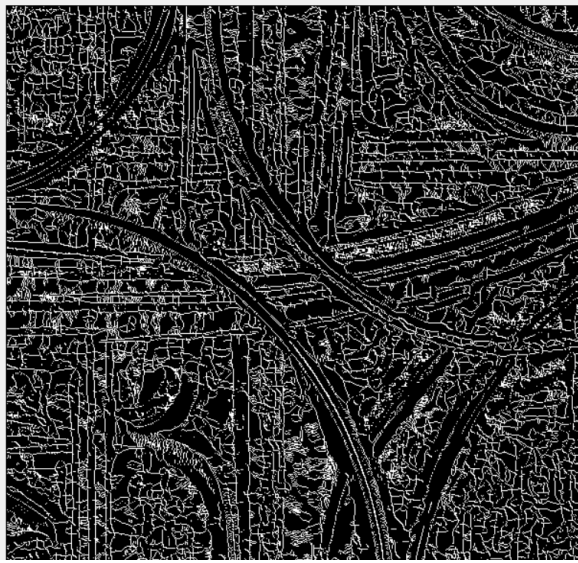


Figure 2: Edge detection with high sigma($\sigma = 3$)

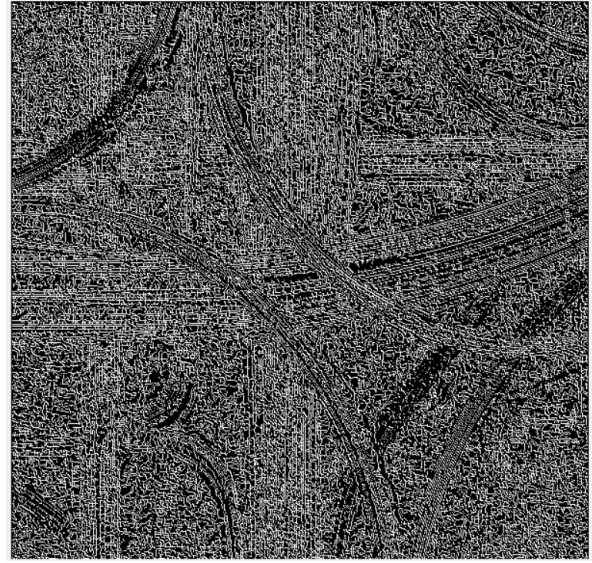


Figure 3: Edge detection with low sigma($\sigma = 1$)

Explanation:

- **Sigma = 1.0** (small sigma): The edges are fine and sharp but noisy.
- **Sigma = 3.0** (large sigma): The edges are smoother and less noisy but may lose fine details.

4.2 Effect of Thresholds

The *low threshold* and *high threshold* values control the detection of strong and weak edges. Lower thresholds result in more weak edges, while higher thresholds result in fewer weak edges being retained.

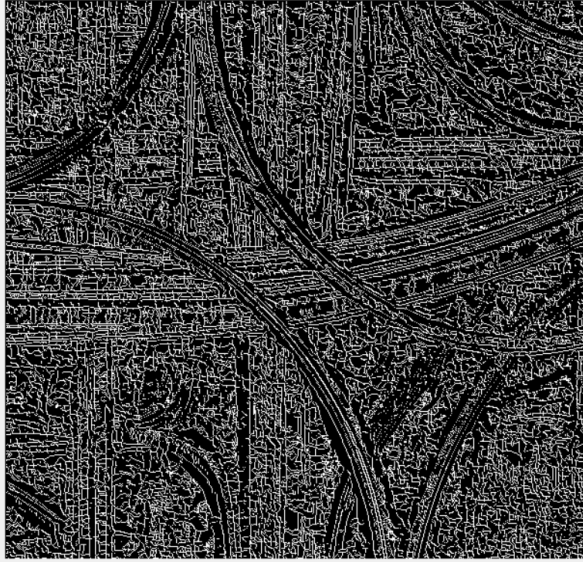


Figure 4: Edge detection with low threshold (Low = 10, High = 50).

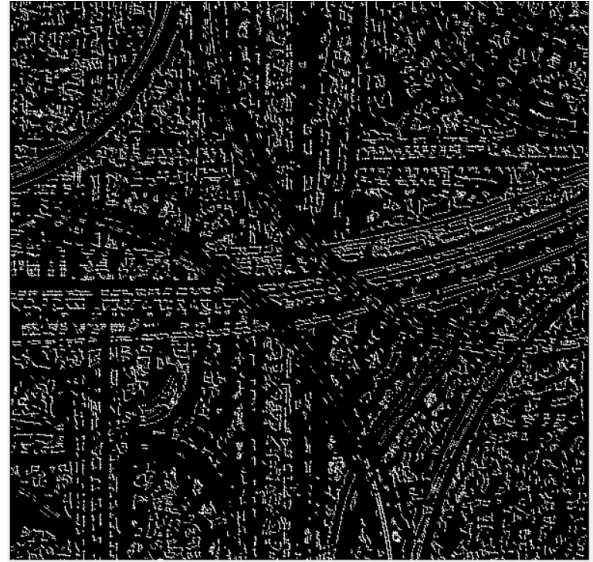


Figure 5: Edge detection with high threshold (Low = 50, High = 300).

Explanation:

- **Low Threshold = 10, High Threshold = 50:** Detects weak and strong edges, potentially noisy.
- **Low Threshold = 50, High Threshold = 300:** Detects only strong edges, cleaner but possibly missing details.

5 Related Edge Detection Methods

While the Canny edge detector is popular, there are other edge detection methods, such as:

- **Sobel Operator:** This method uses simple gradient computation in horizontal and vertical directions to detect edges.
- **Prewitt Operator:** Similar to Sobel, but uses different kernel weights for edge detection.
- **Laplace of Gaussian (LoG):** This method applies a Laplacian filter after smoothing with a Gaussian function to detect edges.

These methods are often simpler but less accurate than the Canny edge detector in terms of detecting fine edges.

6 Conclusion

The Canny edge detection algorithm provides accurate and efficient edge detection by using Gaussian smoothing, gradient calculation, non-maximum suppression, double thresholding, and edge tracking by hysteresis. The parameters *sigma* and the *low* and *high* thresholds are crucial in tuning the algorithm for specific images, such as satellite images, where edge detection is important for feature extraction.

The effects of varying the parameters of the Canny edge detection (i.e., **Sigma**, **Low Threshold**, and **High Threshold**) are as follows:

- **Small Sigma, Low Threshold, High Threshold:** This setting produces *sharper edges* but introduces *more noise*.
- **Large Sigma, Low Threshold, High Threshold:** This configuration results in *more smoothing* with *fewer finer details*, but *stronger edge definition*.
- **Small Sigma, Medium to High Thresholds:** It detects *sharper edges* but may *miss some finer details*.
- **Large Sigma, High Thresholds:** This setting produces *more distinct edges* with *fewer details*.